P3.5-AUTOMATIZACIÓN. EL SERVIDOR QUE SE AUTOGESTIONA

Cristóbal Suárez Abad

ADMINISTRACIÓN DE SISTEMAS OPERATIVOS - 2º ASIR

Contenido

USUARIO DE SERVICIO 3
Linux 3
Windows 4
MISIÓN 1 – “Que no se nos llene el disco” 21
Linux – Limpieza básica 22
Windows – Limpieza 24
MISIÓN 2 – “El servicio fantasma” 25
Linux 26
Windows 27
MISIÓN 3 – “Usuarios zombis” 29
Linux 30
Windows 31
MISIÓN 4 – “EL DÍA DE LA MARMOTA (BACKUP & RESTORE)” 32
Linux 33
Windows 34

*Tras las vacaciones, el administrador principal de **ASIRCorp** se ha ido de baja y ha dejado el sistema "medio automatizado". Pero debes revisar, completar y mejorar la automatización existente para que el sistema funcione perfectamente sin intervención humana. Para ello te propongo una serie de misiones:*

SEGURIDAD:

Recuerda las restricciones de seguridad en la automatización de tareas:

Ejecutar scripts no como root / administrador, sino como usuario de servicio

Ajustar permisos mínimos

Justificar por escrito por qué tu sistema es el más seguro.

CONCEPTO GENERAL DE SEGURIDAD (IMPORTANTE)

Principios aplicados en TODAS las misiones:

- ❌ **No se ejecuta como root / Administrador**
- ✅ **Se usa usuario de servicio:**
  - **Linux → svc_auto**
  - **Windows → svc_auto**
- ✅ **Permisos mínimos:**
  - **Solo lectura/escritura donde sea necesario**
  - **Uso de sudo limitado (Linux)**
- ✅ **Logs centralizados**
- ✅ **Scripts en modo seguro (no destructivos)**

👉 **Justificación:**

Si un script automatizado es comprometido, el impacto queda limitado al mínimo necesario, evitando escaladas de privilegios.

USUARIO DE SERVICIO

Linux

- Crear usuario de servicios

sudo useradd -r -s /bin/bash svc_auto

sudo mkdir /opt/automation

sudo chown svc_auto:svc_auto /opt/automation

```
 Cristobal CON ROOT  jueves  8 enero 2026 11:36
[root@server2asir usuario]$sudo useradd -r -s /bin/bash svc_auto
 Cristobal CON ROOT  jueves  8 enero 2026 11:36
[root@server2asir usuario]$sudo mkdir /opt/automation
 Cristobal CON ROOT  jueves  8 enero 2026 11:36
[root@server2asir usuario]$sudo chown svc_auto:svc_auto /opt/automation
 Cristobal CON ROOT  jueves  8 enero 2026 11:36
[root@server2asir usuario]$
```

- Darle solo permisos sudo solo para servicios:

sudo visudo

svc_auto ALL=(root) NOPASSWD: /bin/systemctl restart apache2

```
# Permisos para svc_auto

svc_auto ALL=(root) NOPASSWD: /bin/systemctl restart apache2
```

Windows

Vamos a crear una cuenta gMSA.

¿Por qué?

Vamos a crear una **gMSA (Group Managed Service Account)**, que consta de **4 fases obligatorias**:

1️⃣ Comprobar requisitos
2️⃣ Crear la clave raíz de KDS (solo una vez por dominio)
3️⃣ Crear la cuenta gMSA
4️⃣ Instalarla y probarla en el DC

👉 **Si una fase falla, NO continúes**

1️⃣ COMPROBAR REQUISITOS (OBLIGATORIO)

1.1 Nivel funcional del dominio

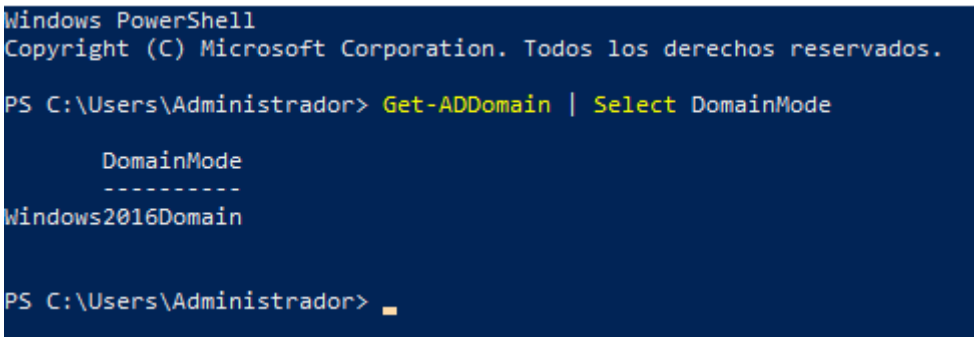
En el DC, abre **PowerShell como Administrador del Dominio** y ejecuta:

Get-ADDomain | Select DomainMode

Debe devolver **al menos**:

Windows2012Domain

✔️ Si es 2012 o superior → OK
❌ Si es inferior → **NO puedes usar gMSA**

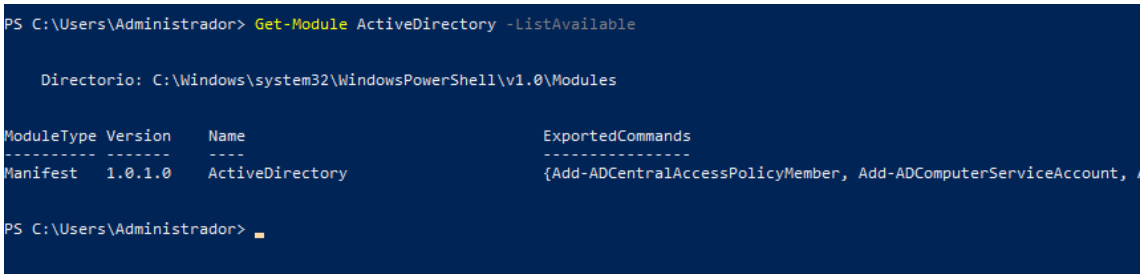


1.2 El DC debe tener el módulo AD

Compruébalo:

Get-Module ActiveDirectory -ListAvailable

Si aparece → OK
Si no → estás en el servidor equivocado



2️⃣ CREAR LA CLAVE RAÍZ KDS (MUY IMPORTANTE)

⚠️ Esto solo se hace UNA VEZ por dominio

⚠️ Si ya existe, NO se repite

Ejecuta:

Get-KdsRootKey

Caso A – No devuelve nada

```
PS C:\Users\Administrador> Get-KdsRootKey
PS C:\Users\Administrador>
```

➡️ Hay que crearla

Add-KdsRootKey -EffectiveImmediately

```
PS C:\Users\Administrador> Add-KdsRootKey -EffectiveImmediately

Guid
----
0926e8e5-731b-6021-32db-8e0fea8aec51


PS C:\Users\Administrador>
```

0926e8e5-731b-6021-32db-8e0fea8aec51

⏳ Nota:

- En entornos reales tarda hasta 10 horas
- En laboratorio funciona inmediatamente

Caso B – Devuelve una clave

➡️ NO hagas nada más

✔️ Continúa

3️⃣ CREAR LA CUENTA gMSA (PASO CRÍTICO)

Ahora sí. Esto crea la identidad, pero no la instala aún.

Ejecuta:

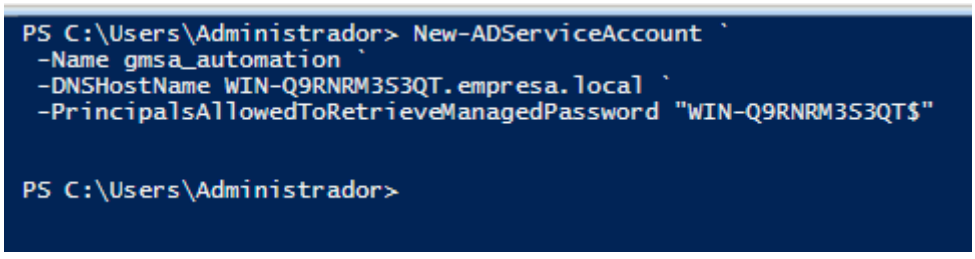
```
New-ADServiceAccount `
 -Name gmsa_automation `
 -DNSHostName dc1.tudominio.local `
 -PrincipalsAllowedToRetrieveManagedPassword "Domain Controllers"
```

📌 **Explicación línea a línea:**

- **-Name gmsa_automation**
  → Nombre de la cuenta de servicio
- **-DNSHostName dc1.tudominio.local**
  → Nombre real del DC
  👉 Debe coincidir exactamente con el FQDN del DC
- **-PrincipalsAllowedToRetrieveManagedPassword "Domain Controllers"**
  → Solo los DC pueden usarla
  → Nadie más puede robar su identidad

En mi caso:

```
New-ADServiceAccount `
 -Name gmsa_automation `
 -DNSHostName WIN-Q9RNRM3S3QT.empresa.local `
 -PrincipalsAllowedToRetrieveManagedPassword "WIN-Q9RNRM3S3QT$"
```



Te dará problemas porque te dice que la clave aun no existe. Usa esto para que piense que ya se creó 10 horas antes.

```
Add-KdsRootKey -EffectiveTime ((Get-Date).AddHours(-10))
```

Después Vuelve a ejecutar lo de antes.

CUANDO FUNCIONE (IMPORTANTE)

Después de que New-ADServiceAccount no devuelva error, hacemos SOLO esto:

Install-ADServiceAccount gmsa_automation

Test-ADServiceAccount gmsa_automation

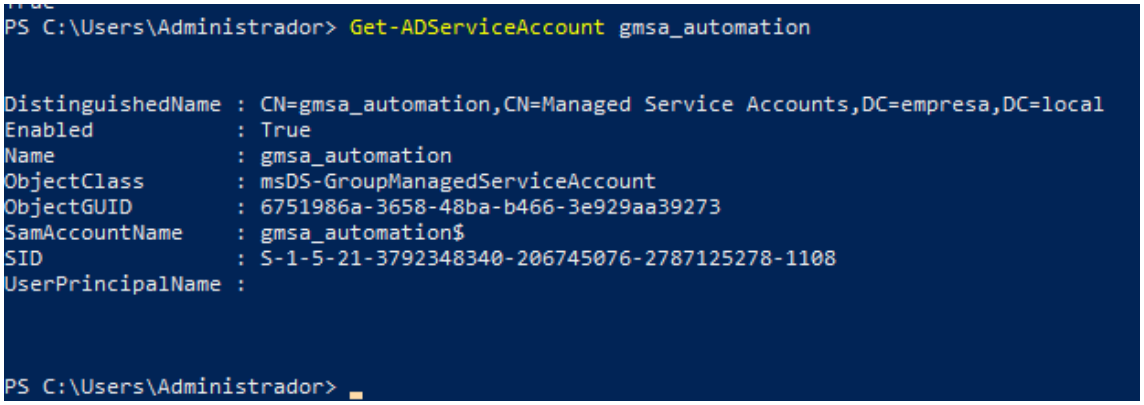
Y debe salir:

True

3.1 Verificar que existe en AD

Get-ADServiceAccount gmsa_automation

Si muestra información → creada correctamente
Si falla → NO continúes



4️⃣ INSTALAR LA gMSA EN EL DC

⚠️ Este paso vincula la cuenta al servidor.

Ejecuta en el DC:

Install-ADServiceAccount gmsa_automation

Luego verifica:

Test-ADServiceAccount gmsa_automation

✔️ Debe devolver:

True

❌ Si devuelve False → algo está mal (no sigas)

5️⃣ CONFIRMAR QUE NO SE PUEDE INICIAR SESIÓN (SEGURIDAD)

Esto es clave para tu tranquilidad.

❌ Intenta (mentalmente, no lo hagas):

- **RDP**
- **Inicio de sesión interactivo**
- **PowerShell runas**

👉 No se puede
La gMSA NO tiene contraseña utilizable

6️⃣ USAR LA gMSA EN EL PROGRAMADOR DE TAREAS

Ahora sí, uso real.

En Task Scheduler:

General

- **Ejecutar como:**

TUDOMINIO\gmsa_automation$

1. Dónde introducir el nombre de la cuenta

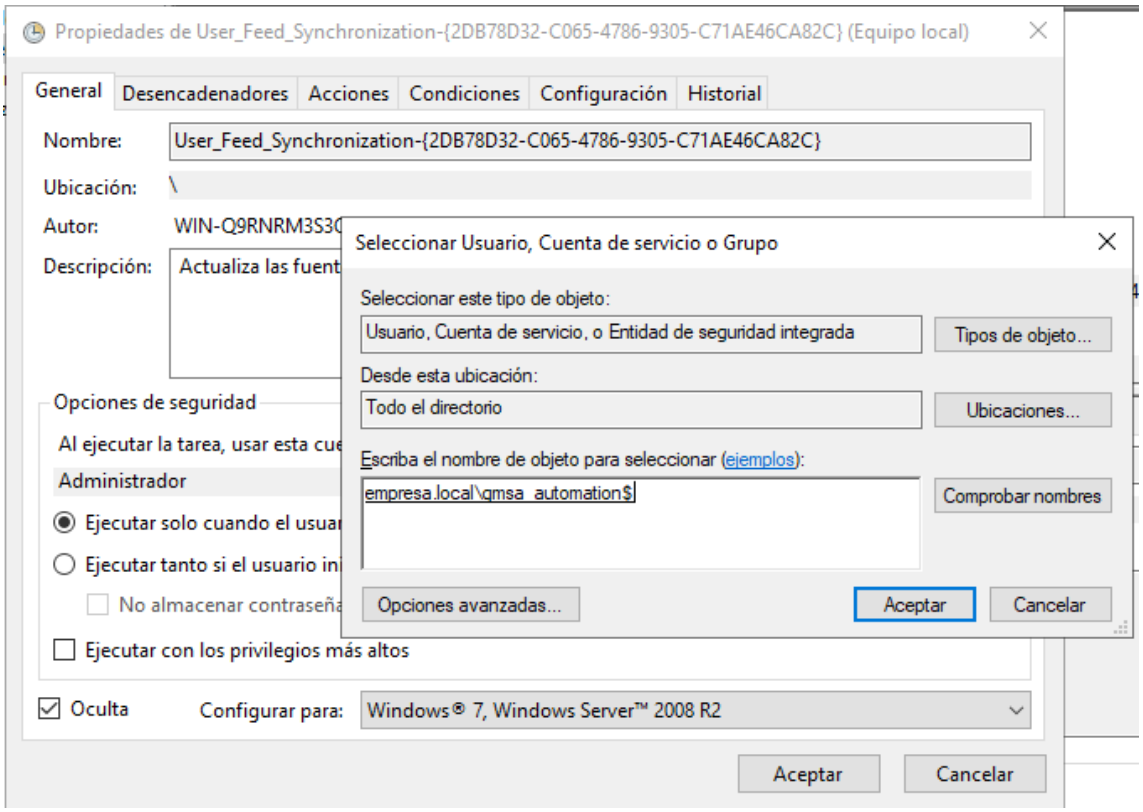
2. Abre el Programador de Tareas y entra en las Propiedades de tu tarea.

3. En la pestaña General, haz clic en el botón Cambiar usuario o grupo....

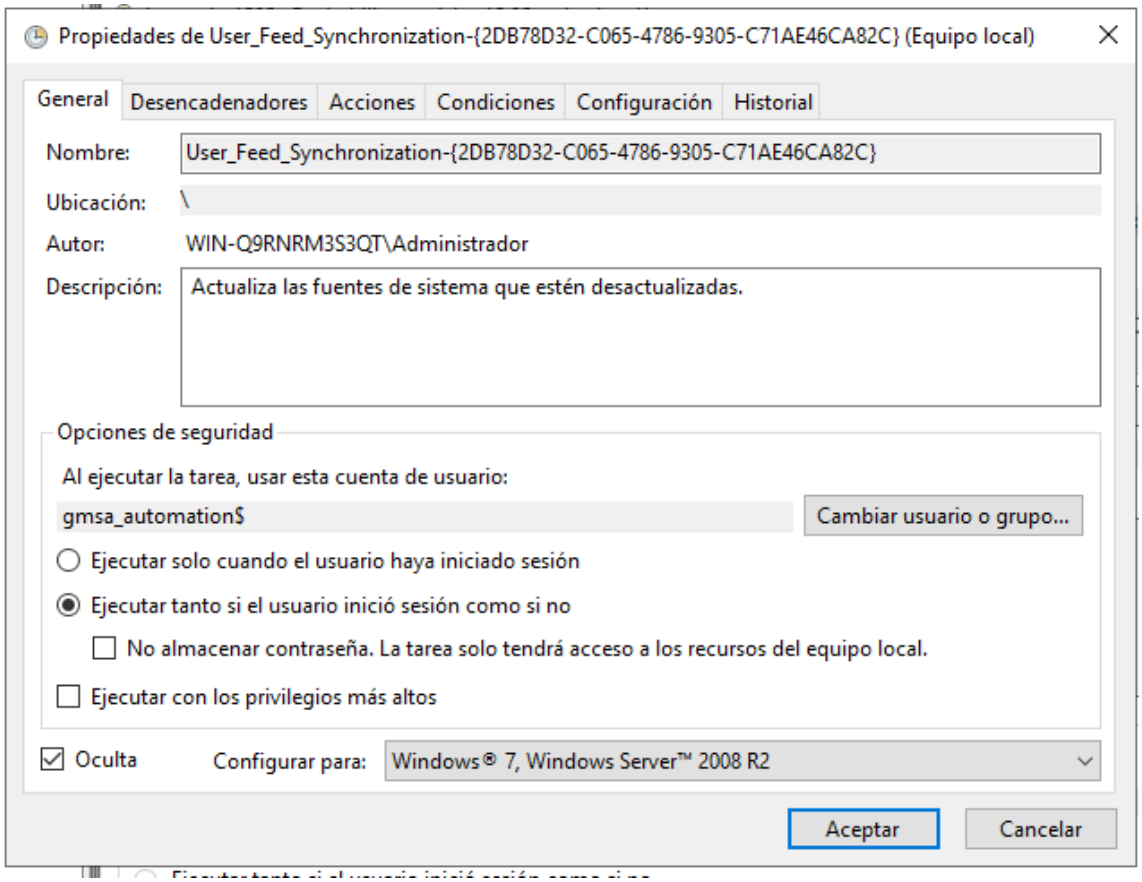
4. Paso Clave: Haz clic en el botón Ubicaciones... y asegúrate de seleccionar tu Dominio (o "Todo el directorio") en lugar del nombre del servidor local.

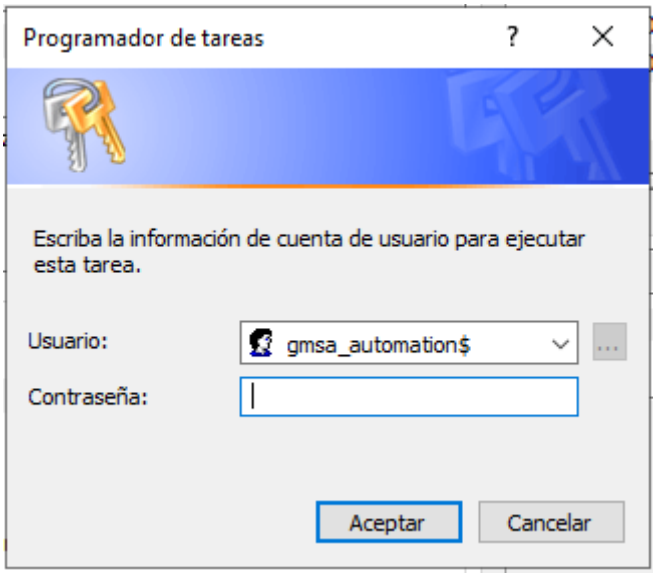
5. Paso Clave 2: Haz clic en el botón Tipos de objetos... y marca la casilla que dice Cuentas de servicio (Service Accounts). Si no marcas esto, Windows no encontrará la gMSA por mucho que escribas el nombre.

6. Ahora escribe el nombre: TUDOMINIO\gmsa_automation$ y pulsa Aceptar.

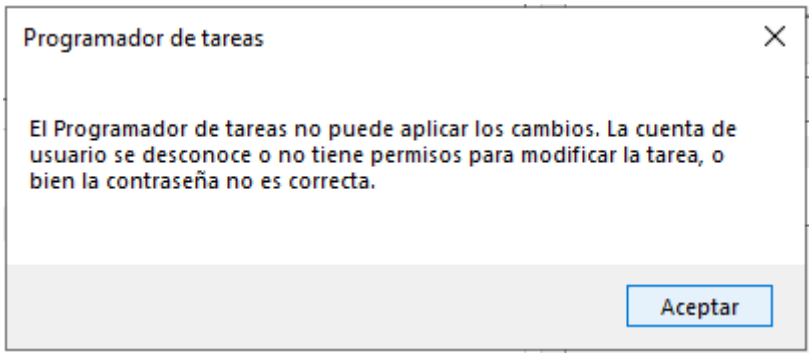


1. Configuración de seguridad obligatoria
2. Una vez que el nombre aparece correctamente en el cuadro "Ejecutar como", configura lo siguiente en esa misma pestaña General:
3. Opción: Selecciona "Ejecutar tanto si el usuario inició sesión como si no".
4. Contraseña: Al darle a "Aceptar" para guardar la tarea, Windows te mostrará un cuadro de diálogo para pedirte la contraseña. Déjalo totalmente vacío y pulsa "Aceptar".
5. Nota: Como es una gMSA, Windows sabe que debe ir al Active Directory a buscar la clave automáticamente. Si intentas poner una contraseña o marcar "No almacenar contraseña", la tarea fallará.


Propiedades de User_Feed_Synchronization-{2DB78D32-C065-4786-9305-C71AE46CA82C} (Equipo local)
General
Desencadenadores
Acciones
Condiciones
Configuración
Historial
Nombre:
User_Feed_Synchronization-{2DB78D32-C065-4786-9305-C71AE46CA82C}
Ubicación:
\
Autor:
WIN-Q9RNRM3S3QT\Administrador
Descripción:
Actualiza las fuentes de sistema que estén desactualizadas.
Opciones de seguridad
Al ejecutar la tarea, usar esta cuenta de usuario:
gmsa_automation$
Cambiar usuario o grupo...
Ejecutar solo cuando el usuario haya iniciado sesión
Ejecutar tanto si el usuario inició sesión como si no
No almacenar contraseña. La tarea solo tendrá acceso a los recursos del equipo local.
Ejecutar con los privilegios más altos
Oculta
Configurar para:
Windows® 7, Windows Server™ 2008 R2
Aceptar
Cancelar


Programador de tareas
Escriba la información de cuenta de usuario para ejecutar esta tarea.
Usuario:
gmsa_automation$
Contraseña:
Aceptar
Cancelar

Puede ocurrir esto:



Ese error es el clásico "muro" al configurar gMSA desde la interfaz visual. Ocurre porque el Programador de Tareas intenta validar la cuenta como si fuera un usuario normal y, al no encontrar una contraseña manual, falla.

Para saltarte este error y configurar la cuenta correctamente, **debes usar PowerShell**. Es la única forma 100% fiable de asignar una gMSA a una tarea.

Sigue estos pasos:

1. Crea la tarea con tu usuario (Borrador)

Crea la tarea de forma normal en la interfaz, con las acciones y disparadores que quieras, pero **déjala configurada con tu usuario actual** por ahora. Guárdala.

2. Ejecuta este comando para "Inyectar" la gMSA

Abre PowerShell como **Administrador** y ejecuta el siguiente bloque (cambiando los nombres por los tuyos):

1. Definir los datos confirmados

$TaskName = "Respaldo_Semanal_Historial"

Importante: Usa el nombre corto de tu dominio (EMPRESA) seguido del nombre con $

$gMSA = "EMPRESA\gmsa_automation$"

2. Crear el objeto de seguridad

$Principal = New-ScheduledTaskPrincipal -UserId $gMSA -LogonType Password -RunLevel Highest

3. Aplicar a la tarea

Set-ScheduledTask -TaskName $TaskName -Principal $Principal

```
PS C:\Users\Administrador> # 1. Definir los datos confirmados
$TaskName = "Respaldo_Semanal_Historial"
# Importante: Usa el nombre corto de tu dominio (EMPRESA) seguido del nombre con $
$gMSA = "EMPRESA\gmsa_automation$"

# 2. Crear el objeto de seguridad
$Principal = New-ScheduledTaskPrincipal -UserId $gMSA -LogonType Password -RunLevel Highest

# 3. Aplicar a la tarea
Set-ScheduledTask -TaskName $TaskName -Principal $Principal

TaskPath                                 TaskName                       State
--------                                 --------                       -----
\                                        Respaldo_Semanal_Historial     Disabled


PS C:\Users\Administrador>
```

Verificación Final

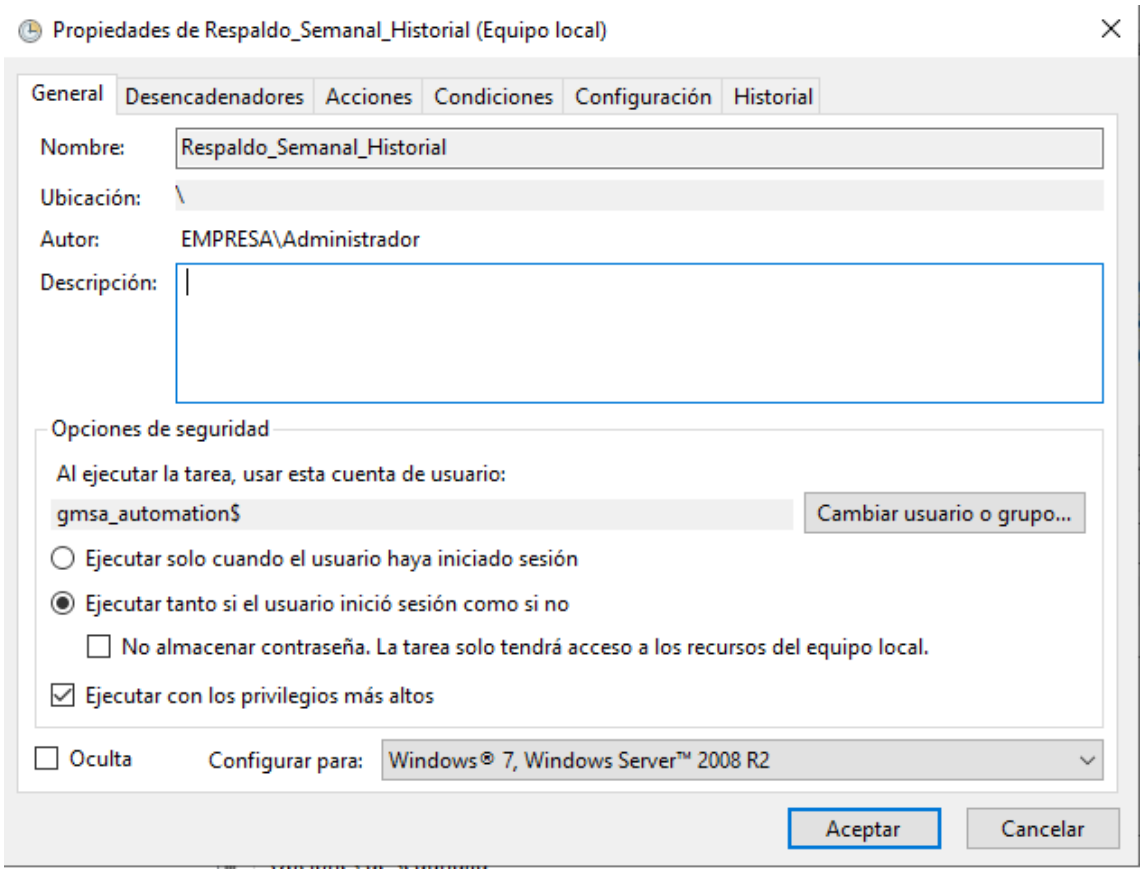
Una vez ejecutado el comando anterior, puedes verificar que la tarea tiene la cuenta asignada sin abrir la interfaz gráfica:

Get-ScheduledTask -TaskName "Respaldo_Semanal_Historial" | Select-Object -ExpandProperty Principal

```
PS C:\Users\Administrador> Get-ScheduledTask -TaskName "Respaldo_Semanal_Historial" | Select-Object -ExpandProperty Principal


DisplayName         :
GroupId             :
Id                  : Author
LogonType           : Password
RunLevel            : Highest
UserId              : gmsa_automation$
ProcessTokenSidType : Default
RequiredPrivilege   :
PSComputerName      :


PS C:\Users\Administrador>
```



⚠️ **El $ es obligatorio**

- ✔ **Ejecutar tanto si el usuario inició sesión como si no**
- ✔ **Ejecutar con los privilegios más altos (solo si es necesario)**

Acción

powershell.exe -ExecutionPolicy Bypass -File C:\Automation\script.ps1

✔ **No se pedirá contraseña**
✔ **No se guarda ninguna credencial**

7️⃣ PERMISOS MÍNIMOS (NO MÁS)

Ahora limitas daños.

Ejemplo:

C:\Automation

├── scripts → Read & Execute

├── logs → Modify

└── backups → Modify

En NTFS:

- **Conceder solo a gmsa_automation$**
- **Quitar Everyone**

1. Aplicar Permisos NTFS (Acceso a Carpetas)

Para que la gMSA pueda leer y escribir sin errores, ejecuta estos comandos. Esto evita usar la ventana de "Seleccionar usuario" que te está dando problemas:

```
$aclPath = "C:\Automation"
$gMSA = "EMPRESA\gmsa_automation$"

# Dar permisos de lectura a la carpeta de scripts
icacls "C:\Automation\scripts" /grant "${gMSA}:(OI)(CI)RX"

# Dar permisos de modificación a logs y backups
icacls "C:\Automation\logs" /grant "${gMSA}:(OI)(CI)M"
icacls "C:\Automation\backups" /grant "${gMSA}:(OI)(CI)M"

# Quitar a 'Everyone' (Todos) para cerrar la seguridad
icacls "C:\Automation" /remove "Everyone"
```

```
PS C:\Users\Administrador> $aclPath = "C:\Automation"
$gMSA = "EMPRESA\gmsa_automation$"

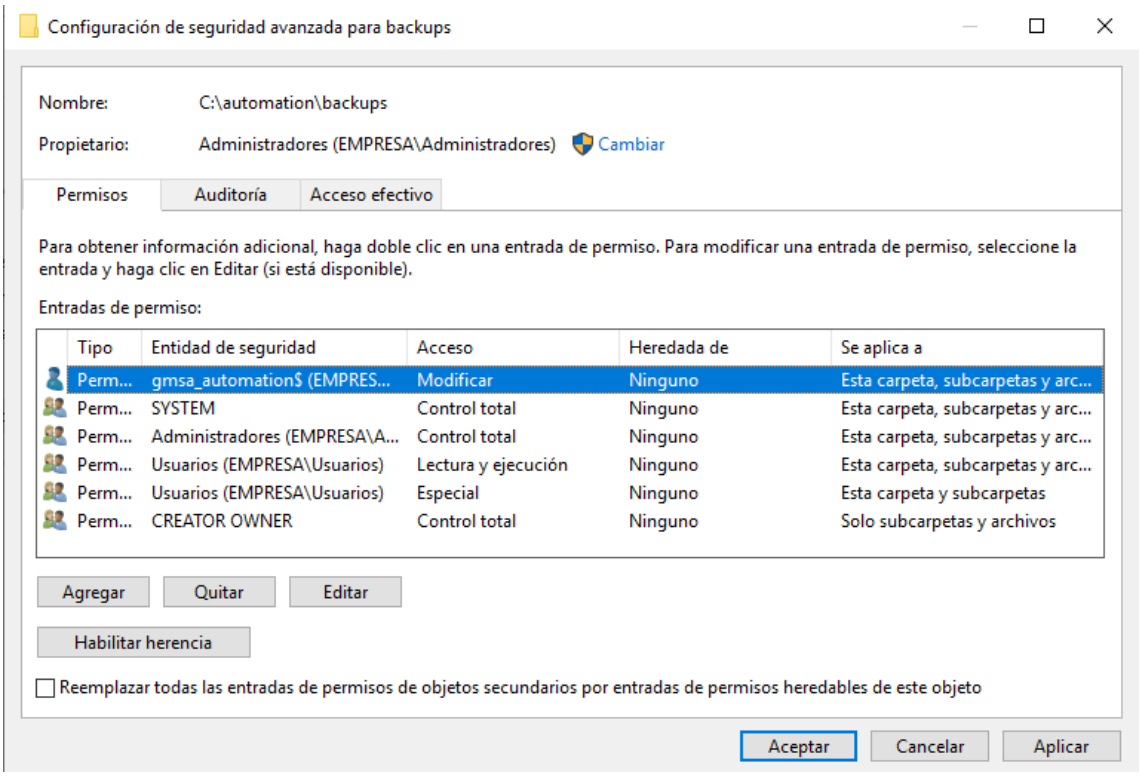
# Dar permisos de lectura a la carpeta de scripts
icacls "C:\Automation\scripts" /grant "${gMSA}:(OI)(CI)RX"

# Dar permisos de modificación a logs y backups
icacls "C:\Automation\logs" /grant "${gMSA}:(OI)(CI)M"
icacls "C:\Automation\backups" /grant "${gMSA}:(OI)(CI)M"

# Quitar a 'Everyone' (Todos) para cerrar la seguridad
icacls "C:\Automation" /remove "Everyone"
archivo procesado: C:\Automation\scripts
Se procesaron correctamente 1 archivos; error al procesar 0 archivos
archivo procesado: C:\Automation\logs
Se procesaron correctamente 1 archivos; error al procesar 0 archivos
archivo procesado: C:\Automation\backups
Se procesaron correctamente 1 archivos; error al procesar 0 archivos
Se procesaron correctamente 0 archivos; error al procesar 0 archivos

PS C:\Users\Administrador>
```

Deshabilitamos herencias manualmente y borramos a quien no queramos.



8️⃣ CHECKLIST FINAL (ESTO TE DA CERTEZA)

Marca mentalmente:

- **Dominio ≥ 2012**
- **KDS Root Key creada**
- **gMSA creada**
- **gMSA instalada**
- **Test devuelve TRUE**
- **No inicio interactivo**
- **Usada solo en tareas**

Permisos mínimos

👉 **Si todo está marcado, es correcto y seguro**

MISIÓN 1 – “Que no se nos llene el disco”

Tengo que automatizar una limpieza básica del sistema

Crea un script que:

- *Limpie /tmp*
- *Comprima logs antiguos*

Programarlo:

- *Linux → cron o systemd timer*
- *Windows → Task Scheduler*

Linux – Limpieza básica

- Script: m1_limpieza.sh (créalo en "/opt/automation/m1_limpieza.sh").

```
#!/bin/bash
LOG="/var/log/automation_limpieza.log"

echo "[$(date)] Inicio limpieza" >> "$LOG"

# Limpiar /tmp (archivos +10 días)
find /tmp -type f -mtime +10 -delete

# Comprimir logs antiguos
find /var/log -name "*.log" -mtime +7 -exec gzip {} \;

echo "[$(date)] Limpieza finalizada" >> "$LOG"
```

- Permisos:

```
chmod 750 /opt/automation/m1_limpieza.sh
chown svc_auto:svc_auto /opt/automation/m1_limpieza.sh
```

- Programación (cron)

```
crontab -u svc_auto -e

0 3 * * * /opt/automation/m1_limpieza.sh
```

Se ejecuta todos los días a las 3:00 AM (Tres de la madrugada).

```
  GNU nano 6.2
#!/bin/bash
LOG="/var/log/automation_limpieza.log"

echo "[$(date)] Inicio limpieza" >> "$LOG"

# Limpiar /tmp (archivos +10 días)
find /tmp -type f -mtime +10 -delete

# Comprimir logs antiguos
find /var/log -name "*.log" -mtime +7 -exec gzip {} \;

echo "[$(date)] Limpieza finalizada" >> "$LOG"
```

```
[root@server2asir usuario]$chmod 750 /opt/automation/m1_limpieza.sh
 Cristobal CON ROOT  jueves  8 enero 2026 11:38
[root@server2asir usuario]$chown svc_auto:svc_auto /opt/automation/m1_limpieza.sh
 Cristobal CON ROOT  jueves  8 enero 2026 11:38
[root@server2asir usuario]$ls -l /opt/automation/m1_limpieza.sh
-rwxr-x--- 1 svc_auto svc_auto 299 ene  8 11:38 /opt/automation/m1_limpieza.sh
 Cristobal CON ROOT  jueves  8 enero 2026 11:39
[root@server2asir usuario]$
```

```
# Automatización de Limpieza de Archivos Temporales

0 3 * * * /opt/automation/m1_limpieza.sh
```

```
[root@server2asir usuario]$crontab -u svc_auto -e
no crontab for svc_auto - using an empty one
crontab: installing new crontab
 Cristobal CON ROOT  jueves  8 enero 2026 11:41
[root@server2asir usuario]$
```

Windows – Limpieza

- Script: m1_limpieza.ps1

Lo creamos en “automation/scripts”

```
$Log="C:\Automation\logs\limpieza.log"

Add-Content $Log "$(Get-Date) Inicio limpieza"

# Limpiar TEMP
Get-ChildItem $env:TEMP -Recurse -ErrorAction SilentlyContinue |
Where-Object {$_.LastWriteTime -lt (Get-Date).AddDays(-10)} |
Remove-Item -Force -Recurse

# Comprimir logs antiguos
$logs="C:\Windows\Logs"
Get-ChildItem $logs -Filter *.log |
Where-Object {$_.LastWriteTime -lt (Get-Date).AddDays(-7)} |
Compress-Archive -DestinationPath "$($_.FullName).zip"

Add-Content $Log "$(Get-Date) Limpieza finalizada"
```

- En el Task Scheduler:

Usuario: svc_auto
Trigger: Diario
Acción: powershell.exe -File -ExecutionPolicy Bypass -File "C:\Automation\scripts\m1_limpieza.ps1"

Debe quedar así:
Iniciar un programa; C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe -ExecutionPolicy Bypass -File "C:\Automation\scripts\m1_limpieza.ps1"

Ahora debemos otorgársela al usuario de servicio.

```
# 1. Definir los datos confirmados
$TaskName = "Limpiar"
# Importante: Usa el nombre corto de tu dominio (EMPRESA) seguido del nombre con $
$gMSA = "EMPRESA\gmsa_automation$"

# 2. Crear el objeto de seguridad
$Principal = New-ScheduledTaskPrincipal -UserId $gMSA -LogonType Password -RunLevel Highest

# 3. Aplicar a la tarea
Set-ScheduledTask -TaskName $TaskName -Principal $Principal
```

MISIÓN 2 – “El servicio fantasma”

Tengo que detectar y corregir problemas con servicios caidos

Crea un script que:

- Compruebe si un servicio está activo (Apache / Spooler).

Si no lo está:

- Lo reinicie
- Registre el evento

Programarlo: Tanto en Windows como en Linux cada X minutos

Linux

cd /opt/automation/

nano m2_servicio.sh

```
#!/bin/bash
SERVICE="apache2"
LOG="/var/log/automation_servicios.log"

if ! systemctl is-active --quiet $SERVICE; then
   echo "[$(date)] $SERVICE caído. Reiniciando..." >> "$LOG"
   sudo systemctl restart $SERVICE
else
   echo "[$(date)] $SERVICE activo" >> "$LOG"
fi
```

- Programación (cron)

crontab -u svc_auto -e

*/5 * * * * /opt/automation/m2_servicio.sh

Cada 5 minutos

Log de errores para cron:

0 3 * * * /opt/automation/m1_limpieza.sh /var/log/m1_limpieza.log 2>&1

*/5 * * * * /opt/automation/m2_servicio.sh >> /var/log/m2_script.log 2>&1

Windows

m2_servicio.ps1

```
$Service="Spooler"
$Log="C:\Automation\logs\servicios.log"

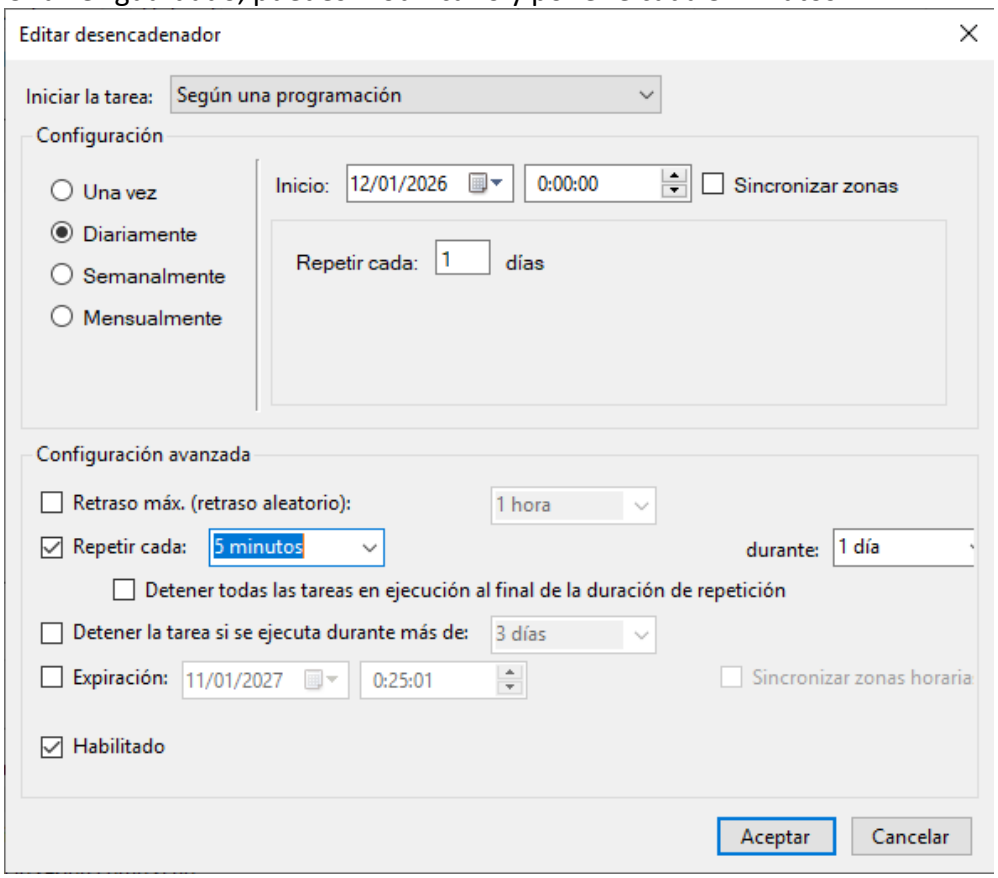
$state = (Get-Service $Service).Status

if ($state -ne "Running") {
   Add-Content $Log "$(Get-Date) $Service detenido. Reiniciando"
   Start-Service $Service
} else {
   Add-Content $Log "$(Get-Date) $Service OK"
}
```

- Ahora el Task Scheduler.

Powershell y opción:
-ExecutionPolicy Bypass -File "C:\Automation\scripts\m2_servicio.ps1"

Una vez guardado, puedes modificarlo y ponerle cada 5 minutos.



Ahora debemos otorgársela al usuario de servicio.

```
# 1. Definir los datos confirmados
$TaskName = "Servicio"
# Importante: Usa el nombre corto de tu dominio (EMPRESA) seguido del nombre con $
$gMSA = "EMPRESA\gmsa_automation$"

# 2. Crear el objeto de seguridad
$Principal = New-ScheduledTaskPrincipal -UserId $gMSA -LogonType Password -RunLevel Highest

# 3. Aplicar a la tarea
Set-ScheduledTask -TaskName $TaskName -Principal $Principal
```

MISIÓN 3 – “Usuarios zombis”

Tengo que repasar las cuentas de usuario

Crea un script que:

- *Liste usuarios inactivos o usuarios con fecha de expiración pasada*

*Programarlo 1 vez al día. Solo **informa**, no borra (modo seguro).*

Linux

cd /opt/automation/

nano m3_usuarios.sh

```
#!/bin/bash
LOG="/var/log/automation_usuarios.log"

echo "[$(date)] Usuarios inactivos:" >> "$LOG"

lastlog | grep "**Never logged in**" >> "$LOG"

echo "-------------------------" >> "$LOG"
```

- Programación (cron)

crontab -u svc_auto -e

Una vez al día

0 1 * * * /opt/automation/m3_usuarios.sh >> /var/log/m3_usuarios.log 2>&1

Windows

m3_usuarios.ps1

```
$Log="C:\Automation\logs\usuarios.log"

Get-LocalUser |
Where-Object {$_.Enabled -eq $true -and $_.PasswordExpires} |
ForEach-Object {
   Add-Content $Log "$(Get-Date) Usuario activo: $($_.Name)"
}
```

- Ahora el Task Scheduler.

Powershell y opción:
-ExecutionPolicy Bypass -File "C:\Automation\scripts\m3_usuarios.ps1"

Ahora debemos otorgársela al usuario de servicio.

```
# 1. Definir los datos confirmados
$TaskName = "Usuarios"
# Importante: Usa el nombre corto de tu dominio (EMPRESA) seguido del nombre con $
$gMSA = "EMPRESA\gmsa_automation$"

# 2. Crear el objeto de seguridad
$Principal = New-ScheduledTaskPrincipal -UserId $gMSA -LogonType Password -RunLevel Highest

# 3. Aplicar a la tarea
Set-ScheduledTask -TaskName $TaskName -Principal $Principal
```

MISIÓN 4 – "EL DÍA DE LA MARMOTA (BACKUP & RESTORE)"

Tengo que pensar en las copias de seguridad. Todo backup debe probarse automáticamente, al menos una vez al día, restaurando un elemento crítico que debe permanecer siempre igual

Crea un script que:

- Realice una copia de seguridad diaria de:
    - Un archivo "congelado" del sistema:
        - Linux → /etc/motd o un archivo de configuración ficticio
        - Windows → un archivo de configuración o un .txt del sistema
- Guarde la copia comprimida con la fecha en el nombre.
- Almacene las copias en un directorio específico de backups.
- Elimine automáticamente las copias antiguas (conservar solo los últimos 5–7 días).
- Restaure automáticamente cada día el archivo "congelado" usando la copia del día anterior.
- Genere un log indicando:
    - Fecha y hora
    - Copia creada
    - Copias eliminadas
    - Archivo restaurado

Programarlo:

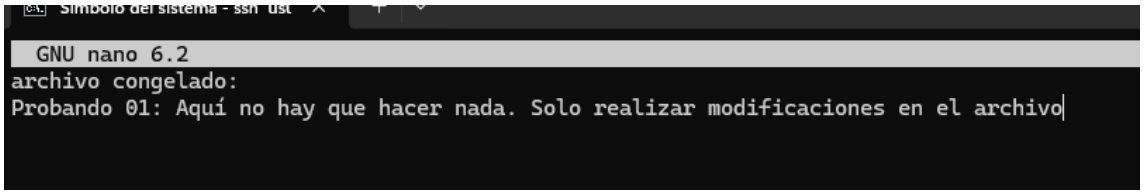
- Linux → cron o systemd timer
- Windows → Task Scheduler

Comprobar que:

- Si se modifica manualmente el archivo "congelado",
  el sistema lo restaura automáticamente al estado original.
- La ejecución queda reflejada en los logs.

Linux

nano /etc/motd



nano /opt/automation/m4_backup_restore.sh

```
#!/bin/bash

SRC="/etc/motd"
BACKUP="/opt/backups"
LOG="/var/log/automation_backup.log"
DATE=$(date +%F)

mkdir -p $BACKUP

# Backup
tar czf $BACKUP/motd_$DATE.tar.gz $SRC
echo "[$(date)] Copia creada: motd_$DATE.tar.gz" >> $LOG

# Eliminar antiguos (>7 días)
find $BACKUP -type f -mtime +7 -delete
echo "[$(date)] Copias antiguas eliminadas" >> $LOG

# Restaurar copia anterior
YESTERDAY=$(date -d "yesterday" +%F)
tar xzf $BACKUP/motd_$YESTERDAY.tar.gz -C /
echo "[$(date)] motd restaurado" >> $LOG
```

- Programación (cron)

crontab -u svc_auto -e

30 2 * * * /opt/automation/m4_backup_restore.sh

##La tarea se ejecuta **todos los días a las 2:30 AM**.

Windows

- Archivo congelado:

C:\Automation\config\system.txt

- Script:

m4_backup.ps1

```
# ============================================================
# MISIÓN 5.1 – GESTIÓN DE SYSTEM.TXT
# ============================================================

$Config = @{
   # Nombre exacto solicitado
   SourceFile  = "C:\Automation\config\system.txt"
   BackupRepo  = "C:\Automation\backups"
   LogFile     = "C:\Automation\logs\backup_system.log"
   Retention   = 7
}

# 1. Preparación de carpetas
foreach ($path in @($Config.BackupRepo, (Split-Path $Config.LogFile))) {
   if (-not (Test-Path $path)) { New-Item -Path $path -ItemType Directory -Force | Out-
Null }
}

function Write-Log {
   param([string]$Message, [string]$Level = "INFO")
   $Stamp = Get-Date -Format "yyyy-MM-dd HH:mm:ss"
   "[$Stamp] [$Level] $Message" | Add-Content $Config.LogFile
}

# --- INICIO DEL PROCESO ---
Write-Log "Iniciando backup de $($Config.SourceFile)"

if (Test-Path $Config.SourceFile) {
   try {
      # 2. Backup con fecha y hora (Ej: system_20240520_1530.zip)
      $Timestamp = Get-Date -Format "yyyyMMdd_HHmm"
      $BackupName = "system_$($Timestamp).zip"
      $DestFile = Join-Path $Config.BackupRepo $BackupName

      Compress-Archive -Path $Config.SourceFile -DestinationPath $DestFile -Force -
ErrorAction Stop
      Write-Log "Copia creada exitosamente: $BackupName"
```

```
    # 3. Limpieza de archivos antiguos
    $LimitDate = (Get-Date).AddDays(-$Config.Retention)
    Get-ChildItem $Config.BackupRepo -Filter "system_*.zip" |
      Where-Object { $_.LastWriteTime -lt $LimitDate } |
      ForEach-Object {
        Remove-Item $_.FullName -Force
        Write-Log "Limpieza: Se eliminó backup antiguo ($($_.Name))" "WARN"
      }

  } catch {
    Write-Log "ERROR al procesar el backup: $($_.Exception.Message)" "ERROR"
  }
} else {
  Write-Log "ERROR: El archivo 'system.txt' no se encontró en la ruta." "ERROR"
  Write-Host "Archivo system.txt no encontrado. Revisa la ruta:
$($Config.SourceFile)" -ForegroundColor Red
}

Write-Log "Fin del ciclo."
Write-Log "--------------------------------------------"
```

- Ahora el Task Scheduler.

Powershell y opción:
-ExecutionPolicy Bypass -File "C:\Automation\scripts\m4_backup.ps1"

Ahora debemos otorgársela al usuario de servicio.

```
# 1. Definir los datos confirmados
$TaskName = "Backup_&_Restore"
# Importante: Usa el nombre corto de tu dominio (EMPRESA) seguido del nombre
con $
$gMSA = "EMPRESA\gmsa_automation$"

# 2. Crear el objeto de seguridad
$Principal = New-ScheduledTaskPrincipal -UserId $gMSA -LogonType Password -
RunLevel Highest

# 3. Aplicar a la tarea
Set-ScheduledTask -TaskName $TaskName -Principal $Principal
```